push_clients (wxtpushclient + jpushclient) 插件深度审查报告

> wxtpushclient v1.1.1 (~1,300 行) + jpushclient v0.0.1 (~1,870 行)
>
> 类型：前端插件 | 合计代码量：~3,170 行
>
> 审查日期：2026-03-27

第一部分：wxtpushclient

综合评分

维度	评分	简评
功能性	7/10	6 厂商覆盖，自动设备识别
安全性	6/10	Token 仅内存存储
架构设计	7/10	多厂商适配器模式合理
接口设计	6/10	平台通道设计清晰但回调不完
错误处理	5/10	部分厂商 SDK 异常未捕获
总评	**6/10**	结构合理，细节不足

核心功能

厂商	支持状态	说明
华为 HMS	✅	HmsMessaging
荣耀 Honor	✅	Honor Push SDK
小米	✅	MiPush SDK
OPPO	✅	OPPO Push
VIVO	⚠️	标记为 "not yet available"
Apple APNs	✅	Flutter APNs

架构分析

```
abstract class PushHandler {
  Future<void> initialize(Map<String, dynamic> config);
  Future<String?> getToken();
```

```
  Future<void> onMessageReceived(Function(PushMessage) callback);
}

class HuaweiPushHandler implements PushHandler { ... }
class XiaomiPushHandler implements PushHandler { ... }
class OppoPushHandler implements PushHandler { ... }
// ...
```

✅ 适配器模式：每个厂商独立实现，可插拔

✅ 平台通道：使用 MethodChannel + EventChannel 与原生通信

✅ 设备自动检测：根据手机品牌自动选择推送 SDK

问题

问题 1：*Token* 无持久化

```
String? _currentToken;

Future<String?> getToken() async {
  if (_currentToken != null) return _currentToken;
  _currentToken = await _channel.invokeMethod('getToken');
  return _currentToken;
}
```

批判：

- Token 仅存在内存中，App 重启 → Token 丢失 → 需要重新获取
- 获取 Token 可能触发网络请求（厂商 SDK 联网注册）
- 应将 Token 持久化到 SharedPreferences/SecureStorage

问题 2：*VIVO* 推送标记 *"not yet available"* 但已注册

```
class VivoPushHandler implements PushHandler {
  @override
  Future<void> initialize(Map<String, dynamic> config) async {
    // TODO: not yet available
    throw UnimplementedError('VIVO push not yet available');
  }
}
```

- 代码中已注册为可用厂商
- VIVO 用户运行到此处会收到未处理异常
- 要么移除注册，要么实现功能

问题 3：无自动重试 *Token* 获取

Token 获取失败（网络异常等）后没有重试机制。首次失败 → 推送永久不可用（直到 App 重启）。

第二部分：jpushclient

综合评分

维度	评分	简评
功能性	5/10	核心推送可用，多个接口残缺
安全性	5/10	无敏感数据处理
架构设计	3/10	巨型单文件，职责混乱
接口设计	3/10	600+ 行防御性类型转换
代码质量	3/10	大量残留代码和 TODO
总评	**3.5/10**	半成品投入使用，技术债高

🔴 致命问题 1：v0.0.1 正在生产使用

```
name: jpushclient
version: 0.0.1  # ← 连 0.1.0 都不是
```

批判：

- 语义化版本中，0.0.x 表示"任何东西都可能在任何时候变"
- 这是一个明确声明"不稳定"的包在生产环境运行
- 无 CHANGELOG、无 README、无文档

🔴 致命问题 2：PushMessage.fromMap() 的 600 行噩梦

```
class PushMessage {
  factory PushMessage.fromMap(Map<dynamic, dynamic> map) {
    // 600+ 行防御性类型转换

    final id = map['id'];
    int? finalId;
    if (id is int) {
      finalId = id;
    } else if (id is String) {
      finalId = int.tryParse(id);
    } else if (id is double) {
      finalId = id.toInt();
    } else {
      finalId = null;
    }

    final title = map['title'];
    String? finalTitle;
    if (title is String) {
      finalTitle = title;
    } else if (title != null) {
```

```
      finalTitle = title.toString();
    }

    // ... 对每个字段重复以上逻辑 × 20+ 个字段
    // ... 共 600+ 行
  }
}
```

批判：

- 这是你能想到的最糟糕的反序列化方式
- 每个字段 10-15 行类型检查代码 × 20+ 字段 = 600 行
- 完全可以用 3 行通用方法替代
- 代码重复率 > 95%

改良方案：

```
class PushMessage {
  factory PushMessage.fromMap(Map<dynamic, dynamic> map) {
    return PushMessage(
      id: _asInt(map['id']),
      title: _asString(map['title']),
      body: _asString(map['body']),
      // ... 每个字段一行
    );
  }

  static int? _asInt(dynamic v) => v is int ? v : v is String ?
int.tryParse(v) : null;
  static String? _asString(dynamic v) => v is String ? v : v?.toString();
}
```

原来 600 行的代码可缩减到 ~30 行。

🔴 致命问题 3：大量残缺接口

```
Future<String?> getAlias() async {
  // TODO: implement
  return null;
}

Future<List<String>> getAllTags() async {
  // TODO: implement
  return [];
}

Future<bool> checkTagBindState(String tag) async {
  // TODO: implement
  return false;
}
```

批判：

- 至少 5 个公共 API 方法返回空值/空列表
- 调用者无法知道返回空是"真的没有数据"还是"功能没实现"
- 应该抛出 `UnimplementedError` 或从接口中移除

🟠 问题 4：巨型 PushService

```
class PushService extends GetxService {
  // 770+ 行！

  // 初始化
  Future<void> init() { ... }

  // 消息处理
  void _onMessageReceived(Map map) { ... }
  void _onNotificationClicked(Map map) { ... }
  void _onNotificationArrived(Map map) { ... }

  // 标签管理
  Future<void> setTags(List<String> tags) { ... }
  Future<void> addTags(List<String> tags) { ... }
  Future<void> deleteTags(List<String> tags) { ... }

  // 别名管理
  Future<void> setAlias(String alias) { ... }
  Future<void> deleteAlias() { ... }

  // 本地通知
  Future<void> showLocalNotification(PushMessage msg) { ... }

  // 消息存储
  void _saveMessage(PushMessage msg) { ... }
  List<PushMessage> getMessages() { ... }

  // ... 还有更多
}
```

- 770 行 GetxService 承担推送接收、UI 通知展示、消息存储、标签管理所有职责
- 强依赖 GetX（`GetxService`），无法在非 GetX 项目中使用

🟠 问题 5：无持久化消息存储

```
final List<PushMessage> _messages = [];  // ← 内存列表

void _saveMessage(PushMessage msg) {
  _messages.add(msg);  // ← App 重启后全部丢失
}
```

推送消息仅存内存列表。用户收到推送 → 杀掉应用 → 再打开 → 推送消息全部消失。

第三部分：两个推送客户端的关系问题

🟠 架构冲突

项目同时存在两个推送客户端插件：

- `wxtpushclient`：自研厂商适配层（华为、荣耀、小米、OPPO、VIVO、Apple）
- `jpushclient`：极光推送封装

问题：

1. 两者功能重叠：都处理推送注册、Token 管理、消息接收
2. 初始化冲突：两个 SDK 同时初始化可能引起冲突
3. Token 注册重复：设备同时向两个推送服务注册 → 可能收到重复推送
4. 职责不清：什么场景用 wxtpushclient，什么场景用 jpushclient?

改良方案：选择一个作为主推送方案：

- 如果主要面向中国市场：保留 `wxtpushclient`（厂商直连，到达率更高）
- 如果需要快速上线：保留 `jpushclient`（极光推送统一平台）
- 无论选择哪个，另一个应该移除或降级为备选方案

改良建议总结

优先级	改良项	涉及插件	预估工作量
P0	决策：选择唯一推送方案	两者	1h 决策
P0	PushMessage.fromMap() 重构为 30 行	jpushclient	2h
P0	移除或标记残缺接口（抛 UnimplementedError）	jpushclient	1h
P1	Token 持久化存储	wxtpushclient	2h
P1	VIVO 推送实现或移除注册	wxtpushclient	4h
P1	拆分 PushService 770 行	jpushclient	6h
P2	推送消息持久化到 SQLite	jpushclient	4h
P2	Token 获取重试机制	wxtpushclient	2h
P3	版本号升级到 ≥0.1.0	jpushclient	0.5h
P3	添加 README 和 CHANGELOG	jpushclient	2h